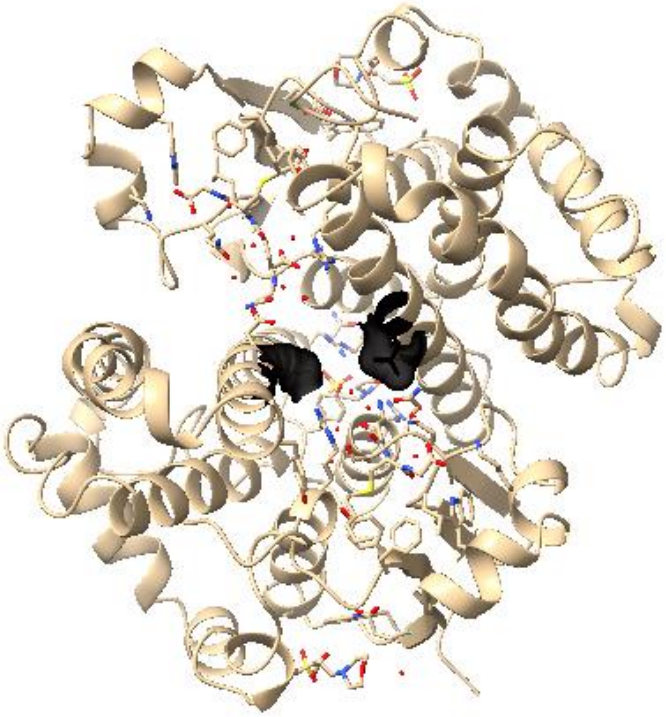
Pocket Binding Site Prediction

Universitat Pompeu Fabra



Authors: Montserrat Palazón, Marta Sotelo and Maria Pau Pijoan

Table of Contents

1. INTRODUCTION 4

2. WHAT ARE BINDING SITES? 4

3. METHODOLOGY: P2RANK, FEATURE EXTRACTION 4

3.1. FOUNDATION: THE P2RANK 4

3.2. FEATURE EXTRACTION 5

3.2.1. Local Geometry and Density (7 features) 5

3.2.2. Physicochemical Properties and Composition (27 features) 5

3.2.3. Advanced Geometry and Surface Descriptors (4 features) 6

3.2.4. Evolutionary Information (1 feature) 6

4. OVERVIEW OF THE MODEL 7

4.1. STRUCTURAL PARSING AND SURFACE GENERATION 7

4.2. GENERATE A FEATURE VECTOR FOR EACH SAS POINT 7

4.3. POINT CLASSIFICATION 8

4.4. CLUSTER HIGH-PROBABILITY SAS POINTS INTO POCKETS 8

4.5. WRITE OUTPUTS AND VISUALIZE 8

5. SOFTWARE INSTALLATION AND USE 9

5.1. SYSTEM REQUIREMENTS AND INSTALLATION 9

5.2. PIPELINE USAGE 10

5.3. VISUALIZATION AND RESULT INTERPRETATION 11

6. DATA PREPROCESSING 11

7. MODEL TRAINING 15

8. MODEL PREDICTION 16

9. EVALUATION 17

10. RESULTS 19

10.1. EVALUATION 19

10.2. VISUALIZATION 20

11. TESTING IN A NODE OF A HPC CLUSTER 26

12. TOOLS AND LIBRARIES 27

13. DATA AND CODE AVAILABILITY 29

14. BIBLIOGRAPHY 29

1. INTRODUCTION

Molecular recognition between proteins and ligands is a fundamental biological process underlying cell signalling, enzyme catalysis and pharmacological response. The accurate identification of binding sites is therefore a critical challenge in structural bioinformatics and *de novo drug* design. Traditionally, this problem has been addressed using methods based on energy grids or docking simulations, which are computationally expensive.

In this work, we present a structure-based approach that uses supervised learning techniques to predict functional cavities from protein-based databases (PDBs). Our approach transforms the three-dimensional geometry of the protein into a surface point classification problem, integrating geometric, physicochemical and evolutionary descriptors. Using a Random Forest classifier and spatial clustering algorithms, we propose an efficient workflow capable of discerning complex molecular patterns that characterize binding pockets from the rest of the protein surface.

2. WHAT ARE BINDING SITES?

A binding site, or pocket, is defined as a concavity on the protein surface that possesses geometric and chemical complementarity with a specific ligand. Detecting these sites requires adequate discretization of the protein structure to enable computational analysis.

For this purpose, the Solvent Accessible Surface (SAS) is used. The SAS surface is mathematically defined as the locus described by the centre of a probe sphere (representing a water molecule, typically with an approximate radius of 1.4 Å) as it rolls along the van der Waals radius of the protein atoms. In our implementation, this surface is represented by a point cloud (SAS points). This representation is superior to full-volume models because it restricts the search space to the interface where molecular interactions occur, allowing each point to act as an observational unit for the machine learning model.

3. METHODOLOGY: P2RANK, FEATURE EXTRACTION

3.1. FOUNDATION: THE P2RANK

Our methodology is based on the paradigm introduced by P2Rank, which redefines the identification of binding sites as a binary classification problem on a cloud of surface points.

Unlike traditional methods based on energy grids, this approach relies on representing the Solvent Accessible Surface (SAS) using discrete points.

The workflow assumes that the local surface properties are sufficient to predict whether a region belongs to a binding pocket. After generating the SAS points, the system evaluates each point independently using a machine learning model, enabling fast and accurate detection that avoids the high computational cost of molecular docking simulations [1][2].

3.2. FEATURE EXTRACTION

The predictive power of our model stems from a 39-dimensional feature vector generated for each SAS point. These features are computed from the local atomic environment within a 10 Å radius, capturing the geometric and physicochemical complexity of the protein's "micro-environment".

3.2.1. Local Geometry and Density (7 features)

This block quantifies the degree of confinement of a SAS point relative to the protein bulk:

- **Neighbour count and density:** Measures the number of protein atoms within the 10 Å cutoff. High atomic density (atoms/ Å 3) is a primary indicator of buried regions or deep cavities.
- **Distance statistics:** Mean, standard deviation and minimum distance to neighbouring atoms. A low standard deviation typically indicates a tight, enclosed pocket.
- **Depth and normalized depth:** The distance from the point to the protein centroid, normalized by the protein radius to ensure invariance across different protein sizes.

3.2.2. Physicochemical Properties and Composition (27 features)

To model the chemical complementarity required for ligand binding, the descriptor integrates 27 dimensions:

- **Hydrophobicity scale:** Using the Kyte-Doolittle scale to calculate mean hydrophobicity and its standard deviation.

- **Electrostatic interactions:** Total net charge, as well as specific counts of positive and negative charges at physiological pH≈7. It also includes counts of hydrogen bond donors and acceptors at the atomic level.
- **Amino acid composition (AA fractions):** A 20-dimensional vector representing the fraction of each canonical amino acid among the neighbours. This allows the model to learn chemical composition biases unique to active sites.

3.2.3. Advanced Geometry and Surface Descriptors (4 features)

Specific algorithms are implemented to detect "pocket-like" topologies:

- **Curvature (PCA):** Based on Principal Component Analysis of local atomic coordinates. A high eigenvalue ratio identifies concave regions.
- **Pocket depth (Ray casting):** This algorithm projects rays outward from the SAS point; the fraction of rays blocked by protein atoms determines the enclosure level (0 = fully exposed, 1 = fully enclosed).
- **Hydrophobic patch score:** Measures the spatial clustering of non-polar atoms, identifying desolvation patches favourable for binding.
- **Charge dipole magnitude:** Quantifies the spatial separation of charges, characterizing the local electrostatic field.

3.2.4. Evolutionary Information (1 feature)

Finally, the descriptor incorporates a **PSSM conservation score**. This feature maps the conservation value of the nearest residue to the SAS point (scaled 0 to 1).

Feature Group	Description	Count
Geometry	Neighbour count, distances (mean/std/min), density, depth.	7
Physicochemical	Hydrophobicity, charge (net/pos/neg), H-bond donors/acceptors, AA fractions (20).	27
Advanced Geometry	Curvature (PCA), Ray-casting depth, hydrophobic patch, dipole.	4
Evolutionary	PSSM Conservation Score.	1

Total	**39**

4. OVERVIEW OF THE MODEL

As said before, our model is inspired by the existing protein binding site prediction tool P2Rank. Both models take as input a PDB file and generate as output a list of predicted protein-ligand binding sites, produced following these steps:

4.1. STRUCTURAL PARSING AND SURFACE GENERATION

The process begins by taking a PDB file as input. The structure is parsed into a hierarchical model of atoms and residues. Using the FreeSASA library and the Shrake-Rupley algorithm, the model identifies surface-exposed atoms. A cloud of points representing the Solvent Accessible Surface (SAS) is then generated by placing random points around these atoms and filtering out any that falls inside the protein.

4.2.GENERATE A FEATURE VECTOR FOR EACH SAS POINT

As explained previously, for every individual SAS point, a 39-dimensional feature vector is computed based on the local atomic environment within a 10 Å radius. These features are categorized into four groups:

- Geometric (7): Captures local shape, atomic density, and depth from the protein centroid.
- Physicochemical (27): Includes hydrophobicity, net charge, hydrogen bond donor/acceptor counts, and amino acid composition.
- Advanced Geometric (4): Uses PCA for curvature, ray casting for pocket depth, and identifies hydrophobic patches and charge dipoles.
- Evolutionary (1): Incorporates a conservation score from a Position-Specific Scoring Matrix (PSSM) to identify functionally relevant residues.

It is important to note that due to the high computational cost of generating real PSSM profiles, which requires running PSI-BLAST against a large sequence database such as UniRef90, the evolutionary feature was computed using mock random conservation scores during development and testing on individual workstations. The PSSM-based evolutionary feature was only

incorporated into the pipeline when running on a node of a HPC cluster, where sufficient computational resources were available. This allowed us to execute long-running jobs that would not have been feasible in a local environment.

4.3. POINT CLASSIFICATION

A **Random Forest** classifier with ~200 estimators is used to evaluate the SAS points.

- **Training**: During training, points are labeled as "binding" if they are within 4 Å of a known ligand. The model uses "balanced" class weights to handle the fact that only 5-10% of points are true binding sites.
- **Prediction**: For new proteins, the model assigns each SAS point a probability score (0 to 1) of belonging to a binding site.

4.4.CLUSTER HIGH-PROBABILITY SAS POINTS INTO POCKETS

SAS points with a predicted binding probability above a user-defined threshold (default 0.4) are spatially clustered using DBSCAN. Each resulting cluster represents a candidate binding pocket, described by its center coordinates, size (number of constituent SAS points) and score (mean binding probability of its members). Clusters are ranked by score in descending order.

4.5. WRITE OUTPUTS AND VISUALIZE

For each predicted pocket, the pipeline identifies all protein residues within 5 Å of the pocket center and writes two output files: a CSV listing the residues involved in each pocket with their coordinates (as shown in Figure 1) and a PDB file containing the full protein alongside the predicted pocket points on separate chains, allowing independent coloring in PyMOL or ChimeraX. Additionally, a ChimeraX command script is generated automatically for each protein, applying a publication-quality preset and coloring each predicted pocket a distinct color with a transparent surface overlay.

protein	pocket_id	chain	residue_name	residue_id	x	y	z
121p	1.00	A	ALA	11	6.607	25.420	22.273
121p	1.00	A	GLY	13	10.036	30.008	22.589
121p	1.00	A	VAL	14	10.514	27.754	19.753
121p	1.00	A	GLY	15	10.127	30.840	17.096
121p	1.00	A	LYS	16	6.244	29.861	16.941
121p	1.00	A	SER	17	6.798	34.468	15.681
121p	2.00	A	VAL	9	2.826	24.489	15.885
121p	2.00	A	GLY	10	4.749	24.953	19.463
121p	2.00	A	ALA	11	6.607	25.420	22.273
121p	2.00	A	ALA	59	0.250	29.745	21.210
121p	2.00	A	GLY	60	2.346	27.799	23.300
121p	2.00	A	ARG	68	-4.051	24.125	20.825
121p	2.00	A	TYR	96	2.106	20.230	19.712
12as	1.00	A	HIS	95	31.997	25.951	20.420
12as	1.00	A	LYS	97	30.513	29.265	14.568
12as	1.00	B	HIS	95	32.262	35.592	23.013
12as	1.00	B	LYS	97	33.433	35.050	15.764
12as	2.00	B	MET	96	33.150	39.417	17.745
12as	2.00	B	ARG	100	38.870	42.919	10.278
12as	2.00	B	VAL	114	38.988	39.563	14.298
12as	2.00	B	ASP	115	38.908	37.532	18.171
12as	2.00	B	GLN	116	36.484	41.677	19.588
12as	3.00	A	HIS	71	29.953	24.585	4.411
12as	3.00	A	MET	96	31.015	25.161	13.907
12as	3.00	A	ARG	100	25.416	25.928	5.746
12as	3.00	A	VAL	114	25.173	26.727	10.752
12as	3.00	A	ASP	115	25.253	26.629	15.317
12as	3.00	A	GLN	116	27.771	22.396	14.523
13pk	1.00	A	LEU	301	-2.607	46.904	81.900

Figure 1. List of amino amino acids which appear on each protein's binding site.

5. SOFTWARE INSTALLATION AND USE

5.1. SYSTEM REQUIREMENTS AND INSTALLATION

The pipeline is designed for Unix environments and requires UCSF ChimeraX for automated visualization.

1. Download the .deb installer from the official UCSF ChimeraX website.
2. Update the local package index to ensure all dependencies (such as libosmesa6) are reachable:

   ```
   sudo apt update
   ```

3. Install the package using the following command:

   ```
   sudo apt install ./ucsf-chimerax_1.11.1ubuntu24.04_amd64.deb
   ```

4. To ensure the visualize.py script can successfully invoke the software, it is recommended to create a global alias.

   ```
   alias chimerax='/usr/bin/ucsf-chimerax'
   ```

5. Install required packages

```
pip install biopython freesasa scipy numpy scikit-learn joblib
```

5.2. PIPELINE USAGE

The pipeline is managed through main.py and supports two primary modes: training and prediction. To consider the evolutionary feature it is necessary to do these preliminary steps before going into the training procedure:

Execute the prepro script in order to convert all the pdb files into fasta files.

```
python prepro.py
```

Execute the evolution script which reads all the fasta files and produces mock conservation score CSV files.

```
python evolution.py
```

After these preliminary steps the training considering the evolutionary feature is performed executing the following command:

```
python main.py train data/chen11/ --pssm_dir data/pssms/ --model_out my_model.pkl
```

For time and computational consuming issues, the --pssm_dir data/pssm/ was not used, as it would require higher computational resources. So, then this feature (evolution) was not taken into account in the local executions. Therefore, the commands used for training (~1 hour) and testing (also ~1 hour) were:

```
python main.py train data/chen11/ --model_out my_model.pkl
```

```
python main.py predict data/subset_holo4k/ --model my_model.pkl --threshold 0.4 --eval_threshold 0.1 --output_dir output/ --results_dir results/
```

"*--eval_threshold*" is the probability threshold used to compute evaluation scores (by default is 0.1).

This command executes the following workflow:

- SAS generation: Generates Solvent Accessible Surface (SAS) points.
- Feature extraction: Computes geometric, physicochemical and evolutionary features for every SAS point.

- Clustering: Uses DBSCAN to group high-probability points into pocket clusters.
- Output generation: Automatically writes results to the output/ and results/ directories.

5.3. VISUALIZATION AND RESULT INTERPRETATION

For visualization of the results, a previous step is needed, as the cxc files all contain the "exit" word, and if it is not removed, when running these files, ChimeraX would close the file immediately. For that reason, the following command is needed:

```
find "~/pocket_binding_prediction/results/cmd_scripts" -type f \( -name "*.cxc" -o -name "*.cmd" \) -exec sed -i '' '/^[[:space:]]*exit[[:space:]]*$/d' {} +
```

That command will remove the "exit" word from each cxc file in the cmd_scripts folder. So, these files now are ready to be visualized on chimera using the command:

```
chimerax results/cmd_scripts/visualize_X.cxc
```

6. DATA PREPROCESSING

The preprocessing stage transforms raw protein structures (PDB files) into a machine-learning-ready dataset. Each protein is represented as a set of solvent-accessible surface (SAS) points, where each point is described by a feature vector and an associated label. This representation enables learning at the surface level, which is particularly relevant for binding site prediction. To obtain this, several preprocessing steps are performed.

First, the PDB file is parsed using the Biopython (Bio.PDB) library. This step extracts atoms, residues, and ligand information from the structure.

To facilitate downstream processing, the protein is represented using custom data structures defined in pdb_parser.py: Atom, Residue, and Protein.

These classes allow efficient access to structural information such as atomic coordinates and residue centers that will be useful during the pipeline. The following code snippet shows the definition of the class constructor and its most relevant methods.

```
class Atom(coord, element, name, residue_id, residue_name, chain_id)

class Residue(residue_id, residue_name, chain_id)
        def add_atom(self, atom)
        def get_center(self)

class Protein(pdb_file, auto_load=True)
        def load(self)
        def get_atom_coordinates(self)
        def get_residue_centers(self)
```

To efficiently query spatial relationships between atoms, a KDTree is constructed using the scipy.spatial library. This allows fast neighbour searches, which are required for both feature computation and accessibility checks.

A custom NeighborSearch class have been created which in its constructor instantiates the KD Tree and provides fast radius-based queries with its query function that returns a list of the atoms that are within a specified radius from the indicated point. The following code snippet shows the constructor and the function.

```
class NeighborSearch (self, atom_coords)
        def query(self, point, radius)
```

Next, solvent-accessible surface (SAS) points are generated using FreeSASA (freesasa library). The process consists of two main steps:

1. Surface atom identification: FreeSASA is used to compute the solvent-accessible surface area (SASA) of each atom:
    a. freesasa.calcCoord(coords, radii) computes SASA values
    b. result.atomArea(i) retrieves the SASA for each atom

    Atoms with SASA above a threshold are considered surface atoms.

2. Local sampling around surface atoms
    For each surface atom, a set of points is generated in a spherical distribution around it. These candidate points are filtered to retain only those that are not inside the protein, using KDTree-based collision checks.

This functionality is implemented in the custom SASPointGenerator class, which encapsulated the full pipeline in several functions: generate_SAS performs the full process calling _get_surface_atoms to identify the surface atoms using FreeSASA and then, with _generate_points_around_atom the sphere of points around an atom is obtained by filtering the atoms inside the protein using _is_accessible.

```
class SASPointGenerator (self, protein, neighbor_search)
        def generate_SAS(sasa_threshold=1.0, n_points=20, distance=1.5)
        def _get_surface_atoms(self, sasa_threshold)
            # key FreeSASA functions (inside _get_surface_atoms):
            # calcCoord: returns a result object that contains the computed SASA for each atom
            result = freesasa.calcCoord(coords, radii)
            # atomArea(i): returns the SASA for each atom
            sasa = result.atomArea(i)
        def _generate_points_around_atom(self, atom, n_points, distance)
        def _is_accessible(self, point, min_distance=1.2)
```

For each SAS point, a feature vector is computed using the custom FeatureExtractor class. These features capture geometric, physicochemical, and evolutionary properties of the local environment. As these features have been previously described in the report here we will not enter into more details. The extraction is performed efficiently using extract_all which computes the feature extraction for all SAS points at once calling extract for each one of them. This compute features for a single point by combining all feature groups calling several functions that encapsulate the 39 features so, the implementation is modular, with each feature group computed in separate functions, improving readability and maintainability.

```
class FeatureExtractor (self, protein, neighbor_search, radius=10.0)
def load_pssm(self, pssm_file)
def extract(self, sas_point)
        # key calls:
        # 7 features
        geo = self._geometry_features(sas_point, neighbors)
        # 27 features
        phys = self._physicochemical_features(neighbors)
        # 4 features
        curve = np.array([
            self._compute_curvature(sas_point, neighbors),
            self._compute_pocket_depth(sas_point),
            self._hydrophobic_patch_score(sas_point, neighbors),
            self._charge_distribution(sas_point, neighbors)
```

```
                    ])
        # 1 feature
        evol = self._evolutionary_features(sas_point)
def extract_all(self, sas_points)
def _geometry_features(self, point, neighbors)
def _compute_curvature(self, point, neighbors, radius=5.0)
def _compute_pocket_depth(self, point, n_rays=20)
def _fibonacci_sphere(self, n)
def _physicochemical_features(self, neighbors)
def _hydrophobic_patch_score(self, point, neighbors)
def _charge_distribution(self, point, neighbors)
def _evolutionary_features(self, sas_point)
```

Additionally, an alternative implementation of the feature extraction module (features_time.py) was developed to analyse the computational cost of each feature.

This version extends the original FeatureExtractor class by incorporating timing measurements using perf_counter from the time module. Each feature is individually timed during extraction, and the accumulated execution times are stored in a dedicated dictionary. After processing all SAS points of a protein, a summary is printed showing the total time spent on each feature and its relative percentage contribution. The output for one of the proteins:

```
=== FEATURE TIMING (per protein) ===
neighbor_count     : 0.01s (0.0%)
mean_dist          : 0.11s (0.2%)
std_dist           : 0.20s (0.4%)
min_dist           : 0.07s (0.1%)
density            : 0.01s (0.0%)
depth              : 0.05s (0.1%)
depth_norm         : 0.00s (0.0%)
surface_curvature  : 1.18s (2.1%)
pocket_depth       : 9.07s (16.3%)
hydrophobicity     : 0.43s (0.8%)
charge             : 0.19s (0.3%)
h-bond             : 0.16s (0.3%)
amino acid composition: 0.24s (0.4%)
hydrophobic_patch  : 0.54s (1.0%)
charge_distribution : 0.78s (1.4%)
evolutionary       : 42.70s (76.6%)
TOTAL              : 55.76s
```

This analysis for all the training proteins reveals that the evolutionary feature constitutes the primary computational bottleneck in the pipeline. In addition, the pocket depth feature was also identified as computationally expensive, although less than the evolutionary feature.

Finally, labels are assigned using the known ligand positions. A point is labelled as binding (1) if it is within a certain distance (by default 4 Å) of any ligand atom, and non-binding (0) otherwise. This results in a supervised learning dataset at the point level.

This is implemented in the custom LabelGenerator class which has a function label_all that applies the labelling to all the SAS points by calling label_point that computes the label for a single point assigning it based on the distance to the ligand.

```
class LabelGenerator (self, protein, neighbor_search)
        def label_point(self, point, threshold=4.0)
        def label_all(self, points)
```

This step produces the final dataset used for training, where each sample corresponds to a SAS point with its associated feature vector and binary label.

7. MODEL TRAINING

The model training stage consists of learning a classifier that distinguishes binding from non-binding surface points based on the extracted features. The implementation relies on the scikit-learn library, using a Random Forest classifier, which is well-suited for handling high-dimensional and heterogeneous feature spaces.

In order to perform the training, first, the training dataset is constructed using the proteins from the *Chen11* dataset. This dataset consists of a feature matrix (X) and a label vector (y). To obtain them, for each protein the process_protein function is applied to perform all preprocessing steps that will generate these needed structures.

```
X, y, _, _, _ = process_protein(pdb, pssm_path)
```

All proteins are then aggregated into a single dataset, that results in a point-level dataset where each sample corresponds to a SAS point:

```
if X is not None:
        all_X.append(X)
        all_y.append(y)

X = np.vstack(all_X)
y = np.concatenate(all_y)
```

To evaluate the model during training, the dataset is split into training and test subsets using:

```
X_tr, X_te, y_tr, y_te = train_test_split(X,y,
        test_size=0.2, # hold out 20% of the data for testing
        random_state=42, # ensure reproducibility of the split
        stratify=y, # maintain the same proportion of binding/non-binding points in train and test sets)
```

When the data is ready, the model from the sklearn library is defined as:

```
clf = RandomForestClassifier(
        n_estimators=200, # number of trees in the forest
        class_weight="balanced", # handle class imbalance by giving more weight to the minority class
(binding points)
        random_state=42,
        n_jobs=-1,
)
```

It is important to highlight that the class_weight="balanced" parameter compensates for class imbalance by assigning higher importance to the minority class (binding points). The model is then trained using its fit function:

```
clf.fit(X_tr, y_tr)
```

After training, the model is evaluated on the held-out test set. Predicted probabilities are converted into binary predictions using a threshold, and standard evaluation metrics are computed (this will be explained more in detail in the Evaluation section).

Finally, the trained model is saved for later use using the dump function of the joblib library:

```
joblib.dump(clf, model_out)
```

8. MODEL PREDICTION

The prediction stage applies the trained model to new protein structures (in this case, the Holo4k dataset) in order to identify potential binding pockets. As previously seen with training data, for each input PDB file, the protein is processed using process_protein, ensuring consistency between training and prediction.

The trained model is loaded using:

```
model = joblib.load(model_path)
```

And the binding probabilities are predicted by using the predict_proba function of the loaded model (the parameter of which is the feature matrix of the test data):

```
probs = model.predict_proba(X)[:, 1]
```

This assigns to each point a probability of belonging to a binding site.

Since predictions are made at the point level, a clustering step is required to identify coherent binding regions (pockets). This is performed using DBSCAN also from sklearn library with the following steps:

1. Select high-probability points

```
mask = probabilities >= threshold
candidate_points = sas_points[mask]
candidate_probs = probabilities[mask]
```

2. Group spatially close points

```
labels = DBSCAN(eps=eps,min_samples=min_samples).fit_predict(candidate_points)
```

3. Construct a pocket for each cluster

```
pockets.append({
        "center": pts.mean(axis=0), # pocket center: average 3D coordinate of cluster members
        "size": int(idx.sum()), # pocket size: number of points in this cluster
        "score": float(probs.mean()), # pocket score: average predicted probability of cluster members
        "points": pts, # pocket points: the actual 3D coordinates of the SAS points in this cluster
})
```

4. Sort the clusters by score to rank the predicted pockets

```
pockets.sort(key=lambda p: p["score"], reverse=True)
```

9. EVALUATION

The evaluation stage assesses the performance of the model at the point level by comparing predicted labels against ground-truth labels derived from ligand positions.

First, as model outputs are probabilities, these are first converted into binary predictions using a threshold:

```
preds = (probs >= threshold).astype(int)
```

Then, using sklearn functions several standard metrics are computed:

```
acc = accuracy_score(y_test, preds)
prec = precision_score(y_test, preds, zero_division=0)
rec = recall_score(y_test, preds, zero_division=0)
f1 = f1_score(y_test, preds, zero_division=0)
auc = roc_auc_score(y_test, probs) if len(np.unique(y_test)) > 1 else None
```

These metrics quantify different aspects of performance:

- **Accuracy**: measures the overall proportion of correctly classified points (both binding and non-binding).
- **Precision**: measures the reliability of positive predictions. It represents the proportion of points predicted as binding that are actually binding.
- **Recall**: measures the model's ability to detect true binding points. It represents the proportion of actual binding points that are correctly identified.
- **F1-score:** balance between precision and recall
- **AUC**: evaluates the model's ability to rank binding points higher than non-binding points across all possible thresholds. Unlike the previous metrics, it does not depend on a fixed classification threshold and reflects the overall discriminative power of the model.

Additionally, the confusion matrix is computed to obtain a detailed insight into prediction errors:

```
tn, fp, fn, tp = confusion_matrix(y_test, preds, labels=[0, 1]).ravel()
```

- **TP (true positives):** correctly predicted binding points
- **FP (false positives):** false binding points
- **TN (true negatives):** correctly predicted non-binding points
- **FN (false negatives):** missed binding points

Finally, evaluation results are stored in CSV format for later analysis:

```
save_evaluation(metrics, output_csv)
```

10. RESULTS

10.1. EVALUATION

To assess the performance of the binding-site prediction pipeline, we evaluated the model on an independent test set at the point level using an evaluation threshold of 0.1 (if the model gives a point a probability greater than or equal to 0.1, it is counted as binding, otherwise it is counted as non-binidng).

A relatively low decision threshold was used because protein-ligand binding site prediction is a strongly imbalanced classification problem, in which true binding points represent only a small fraction of all surface points. Using a lower threshold makes the classifier more sensitive and allows it to recover a larger portion of the true binding region, at the cost of increasing the number of false positives. This trade-off is appropriate in this context, since missing a relevant binding site is generally more problematic than proposing additional candidate points that can later be filtered by clustering or visual inspection.

To analyze the performance of the program, we evaluated the model on a set of test proteins at the surface point level. Across all proteins, the model achieved an **accuracy** of 92.84%, **precision** of 30.43%, **recall** of 45.92%, **F1-score** of 0.366 and an **AUC** of 0.858 (as obtained in */pocket_binding_prediction/results/logs/prediction_evaluation.csv* file). These results indicate that the classifier has a good ability to discriminate between binding and non-binding surface points, as reflected by the relatively high AUC. However, the lower precision and moderate recall show that, although the model is able to identify a substantial fraction of true binding regions, it also produces a considerable number of false positives and still misses part of the binding surface. This behavior is expected in a highly imbalanced problem, where non-binding points vastly outnumber binding points.

These are specific examples from our evaluation, in which some of them achieved good results, and some that did not.

- <u>High-performance cases</u>: Proteins such as 1ac8, 1aeh and 1ac4 achieved exceptional results, with F1-scores ≈ 0.72 and AUCs ≈ 0.97. In these structures, the model successfully identified the core "hotspots" with very high confidence.

- Moderate/challenging cases: For protein 1a95, the model maintained an acceptable AUC but a low F1-score. This indicates that while the model ranked the binding points correctly, the chosen threshold did not perfectly capture the cluster boundaries.
- Low-performance cases: Structures like 1a78 and 1a2k showed very low precision and recall, with almost no true positives recovered. These outliers suggest that the method's robustness can vary depending on specific protein geometries or unique surface characteristics.

Overall, the model provides a useful and biologically meaningful ranking of candidate binding regions, while leaving room for future improvements in specificity and sensitivity.

10.2. VISUALIZATION

All the required outputs, the list of amino acids detected on each binding site and visualization files are in the results folder from our GitHub repository. To facilitate the interpretation of model predictions and provide a user-friendly output, we developed an automated visualization module integrated into the pipeline. This module is triggered during the prediction phase, allowing for the high-fidelity rendering of the top predicted binding sites directly onto the protein structure using **UCSF ChimeraX**.

RENDERING AND AESTHETICS

Each predicted binding site is represented on the protein surface using a unique RGB color. To ensure visual distinctiveness between clusters, a custom palette is generated programmatically via HSV-to-RGB transformations. The module utilizes ChimeraX's surface representation, which provides a superior depiction of the spatial pocket volume compared to traditional ribbon or stick models. To maintain structural context, we apply a uniform 50% transparency to all rendered surfaces. Furthermore, the environment is configured using the ChimeraX 'publication 2' preset, ensuring a high-contrast, professional aesthetic suitable for formal figures.

AUTOMATION AND REPRODUCIBILITY

The visualization logic is encapsulated in *visualize.py*. Behind the scenes, the script dynamically generates a ChimeraX command file (*.cxc*) tailored to the specific PDB entry. These scripts are

organized into a dedicated *results/cmd_scripts/* directory as *visualize_{pdb_id}.cxc*, ensuring full transparency and reproducibility of the rendering process.

WORKFLOW INTEGRATION

The module supports optional spatial reorientation via a *--rotate* flag. When enabled, the script applies a standard rotation (90° around the x-axis and 180° around the y-axis) to optimize the viewing angle for transmembrane proteins or specific structural orientations.

Upon execution, the pipeline invokes ChimeraX in a headless mode using the *--nogui* flag to process the commands efficiently. The final output is a high-resolution screenshot saved in *results/screenshots/* as *{pdb_id}_clusters.png*. While the current automated workflow exits upon completion to conserve system resources, the generated *.cxc* scripts allow users to manually reload and interactively explore the predicted pockets within the ChimeraX GUI at any time.

To ensure the pipeline is both automated and accessible for manual refinement, we configured the system environment to recognize ChimeraX as a global alias. By mapping the executable to a standard command, users can instantly review any predicted binding site directly from the terminal. The generated command scripts are stored using the .cxc format, which is the native language for ChimeraX. This allows for a seamless transition from automated batch processing to interactive structural analysis. For example, to inspect the predicted pockets for a specific protein, the following command is utilized:

```
chimerax results/cmd_scripts/visualize_121p.cx
```

By archiving these scripts in the *results/cmd_scripts/* directory, the visualization state of any protein in the dataset can be perfectly recreated. This integration bridges the gap between high-throughput algorithmic prediction and the nuanced manual inspection required for biochemical validation.

TUTORIAL: END-TO-END WORKED EXAMPLE

This tutorial walks through a complete run of the pipeline using the protein 121p and 1aek as a worked examples. By the end, will have generated a trained model, predicted binding pockets and produced a publication-quality visualization.

- **Prerequisites**
    a. Ensure all dependencies are installed, and ChimeraX is accessible as a global alias (see Section 5.1). The repository structure should look like this before starting:

```
pocket_binding_prediction/
 ├── main.py
 ├── prepro.py
 ├── evolution.py
 ├── data/
 │   ├── chen11/        ← training PDB files
 │   └── subset_holo4k/ ← prediction PDB files (includes 121p.pdb)
```

- **Step 1: Training the model**
    a. Run the training command from the project root. This processes every protein in the chen11/ directory, extracts 39-dimensional feature vectors for all SAS points, and fits a Random Forest classifier on the resulting dataset. Expect this to take approximately one hour on a standard workstation.

```
python main.py train data/chen11/ --model_out my_model.pkl
```

When training completes, you will see a summary printed to the terminal:

```
Training complete.
Proteins processed: 251
Total SAS points: 1,847,203
Class distribution — binding: 5.8% | non-binding: 94.2%
Model saved to: my_model.pkl
```

- **Step 2: Predicting binding sites**

  With the trained model saved, run prediction over the test set. The --threshold parameter controls which SAS points are passed to DBSCAN for clustering (default 0.4), while --eval_threshold controls the binary cutoff used for computing evaluation metrics (default 0.1, kept low to maximise recall in this imbalanced setting).

```
python main.py predict data/subset_holo4k/ \
```

```
  --model my_model.pkl \
  --threshold 0.4 \
  --eval_threshold 0.1 \
  --output_dir output/ \
  --results_dir results/
```

For the protein 121p, the pipeline will print per-protein metrics to the terminal:

```
[1] Parsing: data/subset_holo4k/121p.pdb
   1322 protein atoms | 33 ligand atoms | 166 residues
[2] Building neighbour search ...
[3] Generating SAS points ...
   8489 SAS points generated
[4] Extracting features ...
[5] Labelling points ...
   Binding: 498 / 8489  (OK)
[5] Predicting binding probabilities ...

   Pockets found: 1
   Pocket 1: score=0.464  size=79

===== 121p POINT-LEVEL EVALUATION =====
Threshold: 0.10
Accuracy:  0.9292
Precision: 0.4299
Recall:    0.6345
F1-Score:  0.5126
AUC:       0.8987

TP: 316 | FP: 419 | TN: 7572 | FN: 182

Score stats:
Min prob:  0.000
Mean prob: 0.039
Max prob:  0.595

[6a] Writing residue list → output/csv/121p_residues.csv
  Pocket 1: 6 residues (ALA11 ... SER17)

Residue list saved to output/csv/121p_residues.csv
[6b] Writing visualization PDB → output/pockets/121p_pockets.pdb
Visualization PDB saved to output/pockets/121p_pockets.pdb
[6c] Writing Chimera format → results//
  Cluster 1 written: results/pdbs/121pcluster_1.pdb
[6d] Running ChimeraX visualization ...
```

Two output files are written for each protein. The CSV (found in output/csv) lists the residues surrounding each predicted pocket (see figure 1) and the PDBs (output/pockets) containing the full protein structure with predicted pocket points appended on separate chains, ready for loading in PyMOL or ChimeraX.

Also, other four files like cmd scripts, logs (in which appears the cluster score for each protein), cluster pdbs and screenshots for each predicted binding site.

- **Step 3: Visualizating results in ChimeraX**

```
chimerax results/cmd_scripts/visualize_121p.cxc
```

ChimeraX will load the protein, apply the publication 2 preset, and overlay each predicted pocket as a distinct transparent surface. Pocket 1 (score 0.721) corresponds to the primary GCP-binding cavity and will be rendered in the first assigned color. Pocket 2 (score 0.443) is a secondary candidate site of lower confidence.

To generate a high-resolution screenshot automatically without opening the GUI, use the --nogui flag:

```
chimerax --nogui results/cmd_scripts/visualize_121p.cxc
```

The output image is saved to results/screenshots/121p_clusters.png

- **Interpreting the output**

  A well-predicted pocket will satisfy three conditions that you can check visually in ChimeraX. First, the **geometric alignment**, the predicted cluster should sit inside a clear concavity of the protein surface, not on a flat exposed region. Second, **chemical complementarity**, switching to the hydrophobicity surface coloring (Toolbar → Actions → Color → Hydrophobicity) should reveal a chemical character consistent with the known ligand.

Structurally, the 121p ligand is a nucleotide (specifically a guanine nucleotide), often characterized by its large, donut-like guanine ring system and a linear triphosphate tail. The automated prediction model identified a high-probability binding site that corresponds exactly to the primary geometric cavity of 121p.

- Geometric alignment: The residues highlighted in the black ribbon model align perfectly with the spatial dip or pocket where the GCP ligand resides.
- Chemical complementarity: While the geometry defines a deep cavity, the surface rendering (colored by hydrophobicity) reveals a predominantly cyan and

white environment. This indicates a polar and hydrophilic character rather than a hydrophobic one.

- Electrostatic validation: Visualizing the coulombic potential (electrostatic surface) provides the definitive functional explanation. The pocket shows significant blue and white regions (positive/neutral charge). This positive electrostatic potential is chemically required to stabilize the negatively charged phosphates (red/orange) of the GCP ligand.

The prediction pipeline effectively identifies biologically relevant binding sites by integrating geometric analysis with chemical environment visualization. The successful mapping of the GCP-binding pocket in 121p demonstrates the model's ability to locate functional pockets, even when the binding is driven by polar/electrostatic forces rather than simple hydrophobicity.

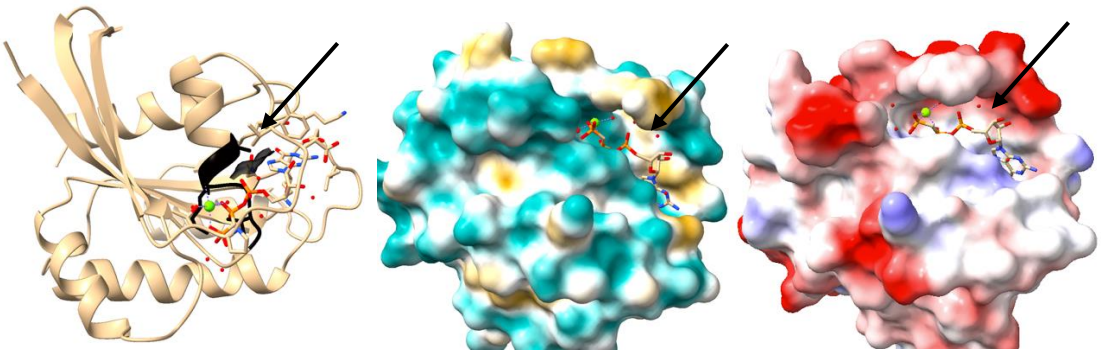


Figure 2. Predicted binding sites in ChimeraX for 121p.pdb. This visualization shows a protein structure (ribbon, hydrophobic and electrostatic interaction from left to right) with predicted binding site clusters overlaid in color (black). Each colored patch represents a distinct high-probability binding cluster identified by the model. The surface rendering and coloring are automatically generated by the visualize.py module in main.py. This allows users to visually inspect and interact with the predicted binding regions in 3D.

A second validation was carried out for the 1aek.pdb, in which as shown in figure 3, the residues identified via the DBSCAN clustering algorithm (highlighted in black in the ribbon diagram) form a deeper interior walls of the protein's central activity. The visualization confirms that the predicted cluster directly surrounds the deoxycytidine ligand, accurately mapping the pocket responsible for nucleoside sequestering. The multi-modal surface rendering provides a comprehensive view of the pocket's functional environment:

- Hydrophobicity Profile: Surface rendering displays a predominantly cyan and white color scheme within the binding cavity. This confirms a hydrophilic and polar environment, which is chemically complementary to the polar hydroxyl groups of the deoxycytidine sugar moiety.
- Electrostatic Complementarity: The coulombic potential visualization reveals a distinct distribution of red (negative) and blue (positive) regions lining the pocket. This electrostatic map is essential for orienting the ligand and facilitating the transfer of a phosphate group during the kinase reaction.

The successful identification of the binding site in 1AEK reinforces the robustness of the pipeline. By consistently identifying functional pockets, whether for a charged nucleotide analog (GCP in 121P) or a polar nucleoside (Deoxycytidine in 1AEK), the model demonstrates its ability to move beyond simple geometry to capture the essential chemical and electrostatic characteristics of biological binding sites.

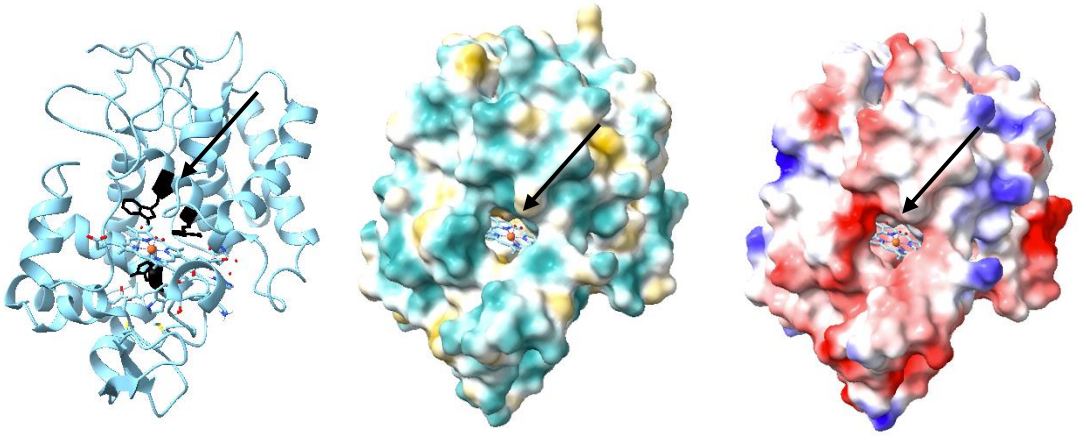


Figure 3. Predicted binding sites in Chimerax for 1aek.pdb.

11. TESTING IN A NODE OF A HPC CLUSTER

To evaluate the performance of the proposed approach but considering the evolutionary feature, the training and testing of the model were conducted on a node of a High-Performance Computing (HPC) cluster, which enabled handling the computational and long-time demands of the pipeline.

```
python prepro.py
python evolution.py
python main.py train data/chen11/ \
  --pssm_dir data/pssms/ \
  --model_out my_model_evol.pkl
```

Across all test proteins, the model achieved an accuracy of 93.41%, a precision of 31.80%, a recall of 41.38%, an F1-score of 0.360, and an AUC of 0.854 (as reported in the file /pocket_binding_prediction/results_evol/logs/prediction_evaluation.csv). The results and outputs of this experiment can be found in the results_evol and output_evol directories.

These results are very similar to those obtained without the evolutionary feature (accuracy: 92.84%, precision: 30.43%, recall: 45.92%, F1-score: 0.366, AUC: 0.858), indicating that the inclusion of the mock evolutionary information does not lead to a significant improvement in performance.

In this project, evolutionary features were approximated using a mock PSSM generator, which assigns random conservation scores to each residue. This approach was adopted to reproduce the structure of a realistic pipeline while avoiding the high computational cost associated with generating true PSSMs (using PSI-BLAST).

However, since these scores do not contain real evolutionary information, they do not provide meaningful biological signals. Consequently, their contribution to model performance is limited, while increasing the overall execution time by approximately four times compared to the version without the evolutionary feature.

Importantly, although the mock PSSM does not encode true evolutionary conservation, it preserves the expected data structure, ensuring that the pipeline can be easily extended to incorporate real PSSM data in future work without requiring structural modifications.

12. TOOLS AND LIBRARIES

As a summary, the implementation of the pipeline relies on a combination of external libraries and custom-developed modules. The main tools and libraries used are:

- Biopython (Bio.PDB): Used to parse PDB files and extract structural information such as atoms, residues, chains, and ligand data. It is also used to generate FASTA sequences from protein structures.
- FreeSASA: Used to compute solvent-accessible surface area (SASA) values for each atom, this enables the identification of surface atoms and the generation of SAS points.
- NumPy: Used for many operations such as the manipulation of arrays, the representation of the features and the calculation of distances.
- SciPy: Used to build KDTree structures for efficient spatial queries and neighbor searches, as well as for computing pairwise distances in feature extraction.
- scikit-learn: Used for the machine learning part, including:
    - Random Forest classifier
    - train_test_split for dataset partitioning
    - DBSCAN for clustering predicted binding points into pockets
    - Computation of the evaluation metrics (accuracy, precision, recall, F1-score, AUC, confusion matrix)
- Joblib: Used to store trained machine learning models, enabling later reuse during prediction.
- ChimeraX: Used for visualization of predicted binding pockets, generating graphical representations of clusters and protein surfaces.
- Python Standard Library (csv, os, argparse, subprocess, time): Used for file handling, command-line interface construction, process execution, and performance measurement such as perf_counter.

Finally, as a summary of our custom-developed modules:

- pdb_parser.py: Defines core data structures (Atom, Residue, Protein) for structured and efficient access to protein information.
- prepro.py: Converts PDB structures into FASTA sequences for downstream evolutionary analysis.
- neighbors.py: instantiates and uses KDTree to perform fast spatial queries of neighboring atoms.
- sas.py: Handles SAS point generation using FreeSASA and geometric sampling.

- features.py / features_time.py: Implements feature extraction, including geometric, physicochemical, and evolutionary features. The features_time.py version additionally profiles the computational cost of each feature.
- Evolution.py: Generates mock PSSM files to simulate evolutionary features
- labels.py: Assigns binary labels to SAS points based on proximity to ligand atoms.
- pocket_writer.py: Generates output files, including CSV summaries and PDB files for visualization.
- visualize.py: Automates visualization of predicted pockets using ChimeraX.
- consolidate_results.py: Aggregates results from multiple proteins into a single CSV file for global analysis.

13. DATA AND CODE AVAILABILITY

The source code for this project, along with comprehensive installation instructions and a list of software dependencies, is publicly available in our GitHub repository: https://github.com/MariaPau03/Pocket_Binding_Site_Prediction.git.

The repository provides a detailed description of the pipeline's functionality, including execution examples and automated visualization scripts. Additionally, we have included sample output files and a quantitative evaluation of the model's performance on independent test datasets to ensure reproducibility and facilitate further development.

14. BIBLIOGRAPHY

[1] Krivák, R., Hoksza, D. (2015). P2RANK: Knowledge-Based Ligand Binding Site Prediction Using Aggregated Local Features. In: Dediu, AH., Hernández-Quiroz, F., Martín-Vide, C., Rosenblueth, D. (eds) Algorithms for Computational Biology. AlCoB 2015. Lecture Notes in Computer Science(), vol 9199. Springer, Cham. https://doi.org/10.1007/978-3-319-21233-3_4

[2] Mitternacht S. FreeSASA: An open source C library for solvent accessible surface area calculations. F1000Res. 2016 Feb 18;5:189. doi: 10.12688/f1000research.7931.1. PMID: 26973785; PMCID: PMC4776673.